

Task1

- 这次是出师未捷身先死，光顾着敲代码，结果弄完显示无法编译，问ai排查问题更是半天弄不出来，最后终于发现是根目录没设置好，以后要打牢地基

关于题目：复制，为了不用再写一遍形成原模板a所写代码，同时以此作为模板建立新的对象b，后续改动b不影响a

结尾用getInfo():因为个人信息，我前面用到了private（防止现实中随意获取篡改个人信息），如普通打印，电脑可能会拒绝

(1) this：用于区分属性和参数，防止电脑搞混，我在上传代码中也有注释

另：如不想用this，可以int a, int b;然后name = a, age = b;但这样可读性变低，所以一般还是用this

(2) 类class：一般写在一段代码开头（至少目前接触到的都是），是一种抽象概念，规定了某东西的性质、定义等，例如Person，规定了人要有姓名性别年龄。有的是系统自带（String），有的要人为规定（Person）。

对象Object：是一个具体东西，参照类的要求生成，如这段代码中写出的小明，18，1

Task2



(3)

- private：只能在本类内部访问，用来修饰name、age等属性，保障数据不被随意篡改，保护安全性
- public：在任何地方都能访问，修饰方法，为了让外部Main可以调用
- protected：同包均可看，不同包子类可看（子类：用extends继承原有东西/再写一遍 后加入新东西）
- 默认：同包可访问，不同不可

(4)

- 用static：使得属于类本身，不依赖具体对象，不读取每个对象的个性化区分
- 这种如getInfo()的普通方法，需要具体对象的数据，即为对象名.方法名（）而不是类名.方法名（）

关于 return :

Java 里的 return 要看方法定义的类型：声明了 String 就要返回字符串（如 getInfo() 里拼接的姓名年龄），声明了 void（如 main）就不用返回值。这和 C 语言里 main 必须写 return 0; 不一样。

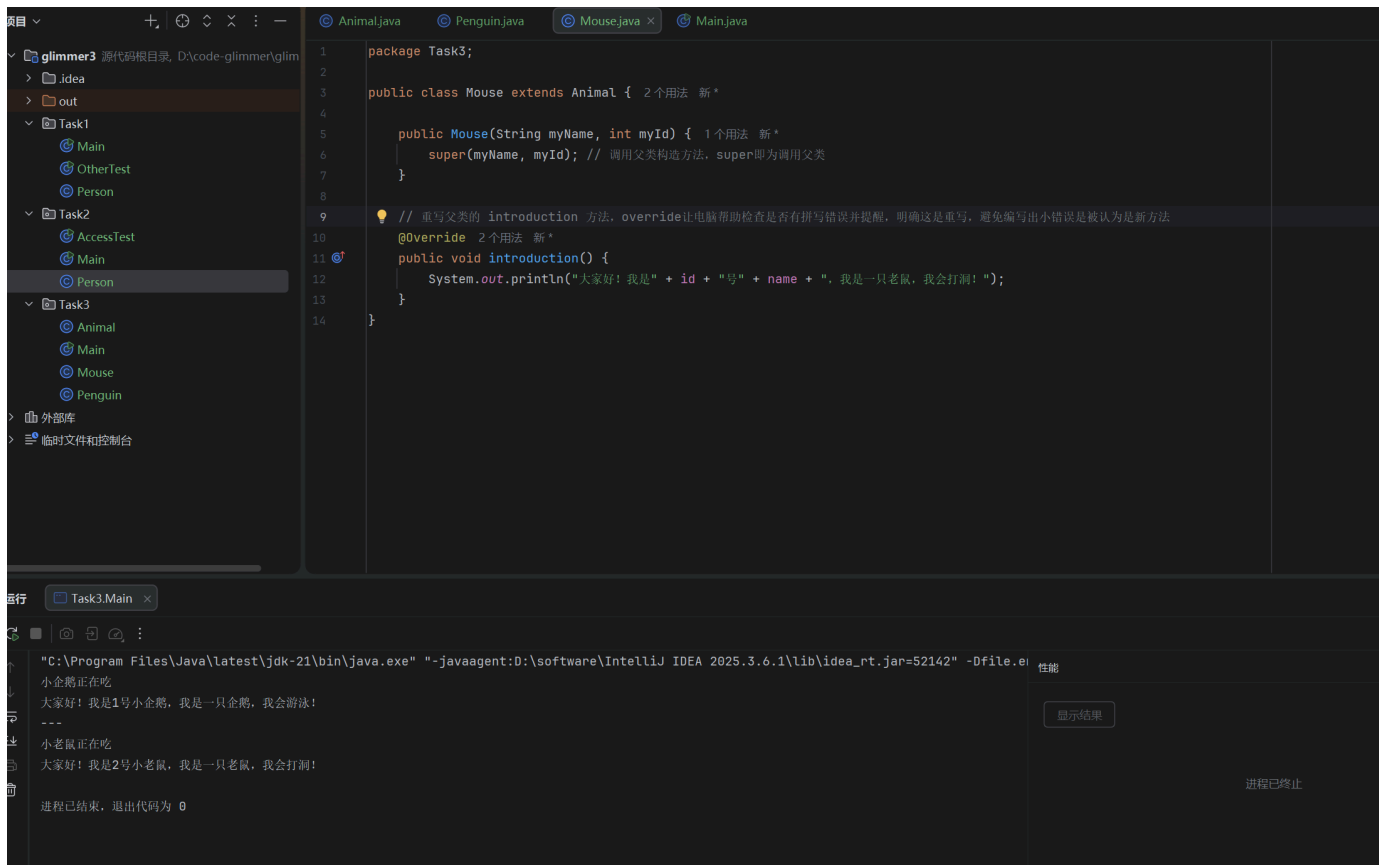
关于 count :

count 前面加了 static，是为了让它属于整个类，所有对象共享同一个计数器。每 new 一个对象，构造方法里的 count++ 就会让它加 1，从而统计一共创建了多少个对象。它和 name、age 不同，后者是每个对象独有的，所以不用加 static。

Task3

- 我先前在task2包里设置了privat，现在task3运行被阻拦了，可是task2里那几个试错文件我不想删，就注释掉了

main函数里有一堆penguin。大写是类名，小写是具体对象，企鹅，new是创建，这里是创建一个名叫小企鹅，编号是一的企鹅实体



```
package Task3;

public class Mouse extends Animal { 2个用法 新 *

    public Mouse(String myName, int myId) { 1个用法 新 *
        super(myName, myId); // 调用父类构造方法，super即为调用父类
    }

    // 重写父类的 introduction 方法，override让电脑帮助检查是否有拼写错误并提醒，明确这是重写，避免编写出小错误被认为是新方法
    @Override 2个用法 新 *
    public void introduction() {
        System.out.println("大家好！我是" + id + "号" + name + "，我是一只老鼠，我会打洞！");
    }
}
```

运行 Task3.Main x

```
"C:\Program Files\Java\latest\jdk-21\bin\java.exe" "-javaagent:D:\software\IntelliJ IDEA 2025.3.6.1\lib\idea_rt.jar=52142" -Dfile.encoding=UTF-8

小企鹅正在吃
大家好！我是1号小企鹅，我是一只企鹅，我会游泳！
---
小老鼠正在吃
大家好！我是2号小老鼠，我是一只老鼠，我会打洞！

进程已结束，退出代码为 0
```

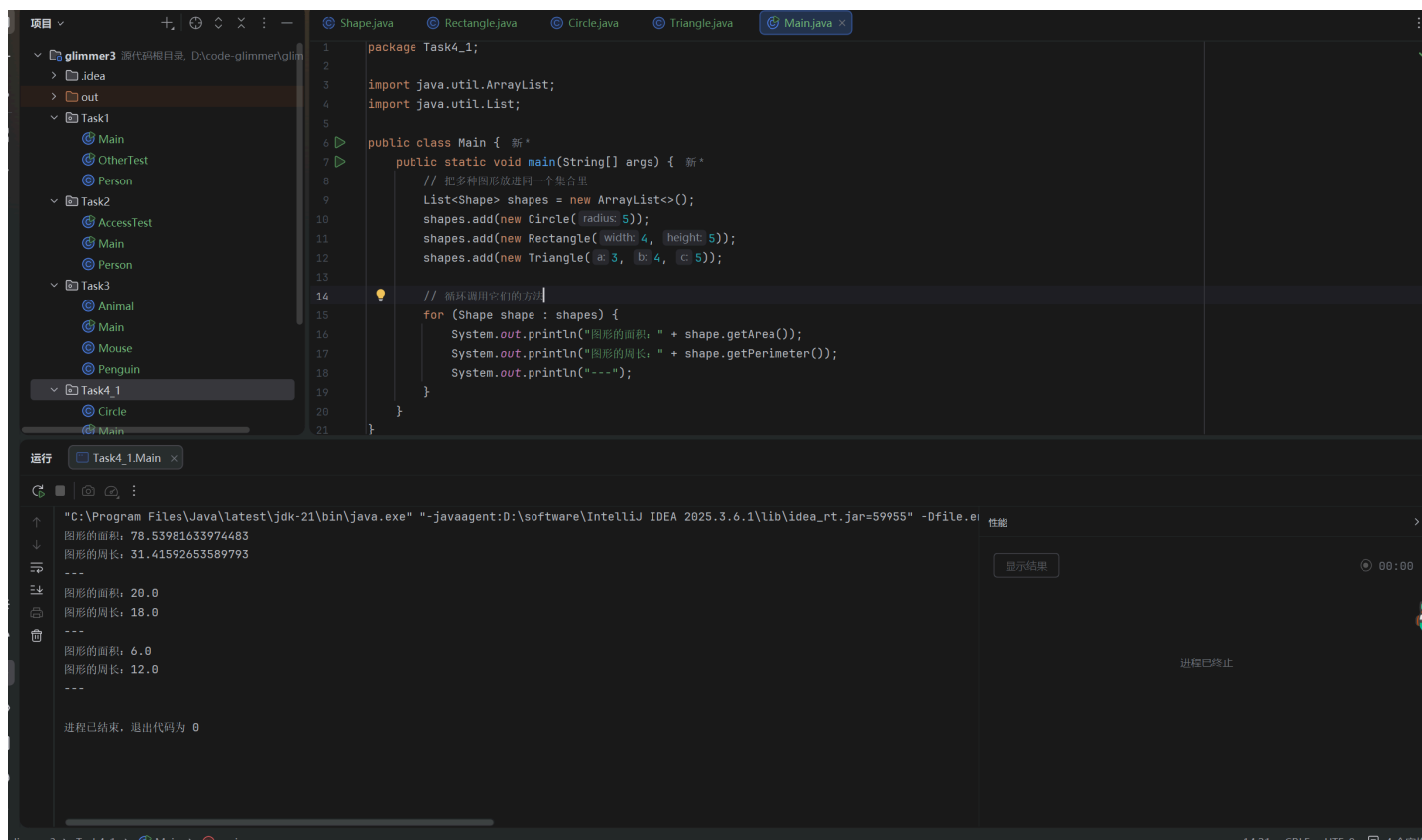
(4)不支持多类继承，会产生菱形冲突

- 多继承，就是一个子类同时继承多个父类，电脑无法判断到底按哪个走
eg：有一个父类叫 Animal（动物），里面定义了方法 eat()，输出“动物在吃东西”。
有两个子类，Dog继承了 Animal，把 eat() 重写为“狗在啃骨头”；Cat也继承了 Animal，把 eat() 重写为“猫在吃鱼”。
现在，如果允许一个类 DogCat同时继承 Dog 和 Cat，那么当 DogCat 去调用 eat() 时，电脑就不知道它到底该执行 Dog 的“啃骨头”，还是 Cat 的“吃鱼”

区分多态（允许）,使程序更简洁:可以多个动物吃骨头，因为他们具体行为姿态不同，但一个动物不能又吃骨头又吃鱼，有点像函数里x和y的关系

Task4

关于题目：多态存在使得多子类可以放在一个集合，允许一个父指向多个子，统一处理，不用分开放置



```
package Task4_1;

import java.util.ArrayList;
import java.util.List;

public class Main {

    public static void main(String[] args) {
        // 把多种图形放进同一个集合里
        List<Shape> shapes = new ArrayList<>();
        shapes.add(new Circle( radius: 5));
        shapes.add(new Rectangle( width: 4, height: 5));
        shapes.add(new Triangle( a: 3, b: 4, c: 5));

        // 循环调用它们的方法
        for (Shape shape : shapes) {
            System.out.println("图形的面积: " + shape.getArea());
            System.out.println("图形的周长: " + shape.getPerimeter());
            System.out.println("----");
        }
    }
}
```

运行 Task4_1.Main

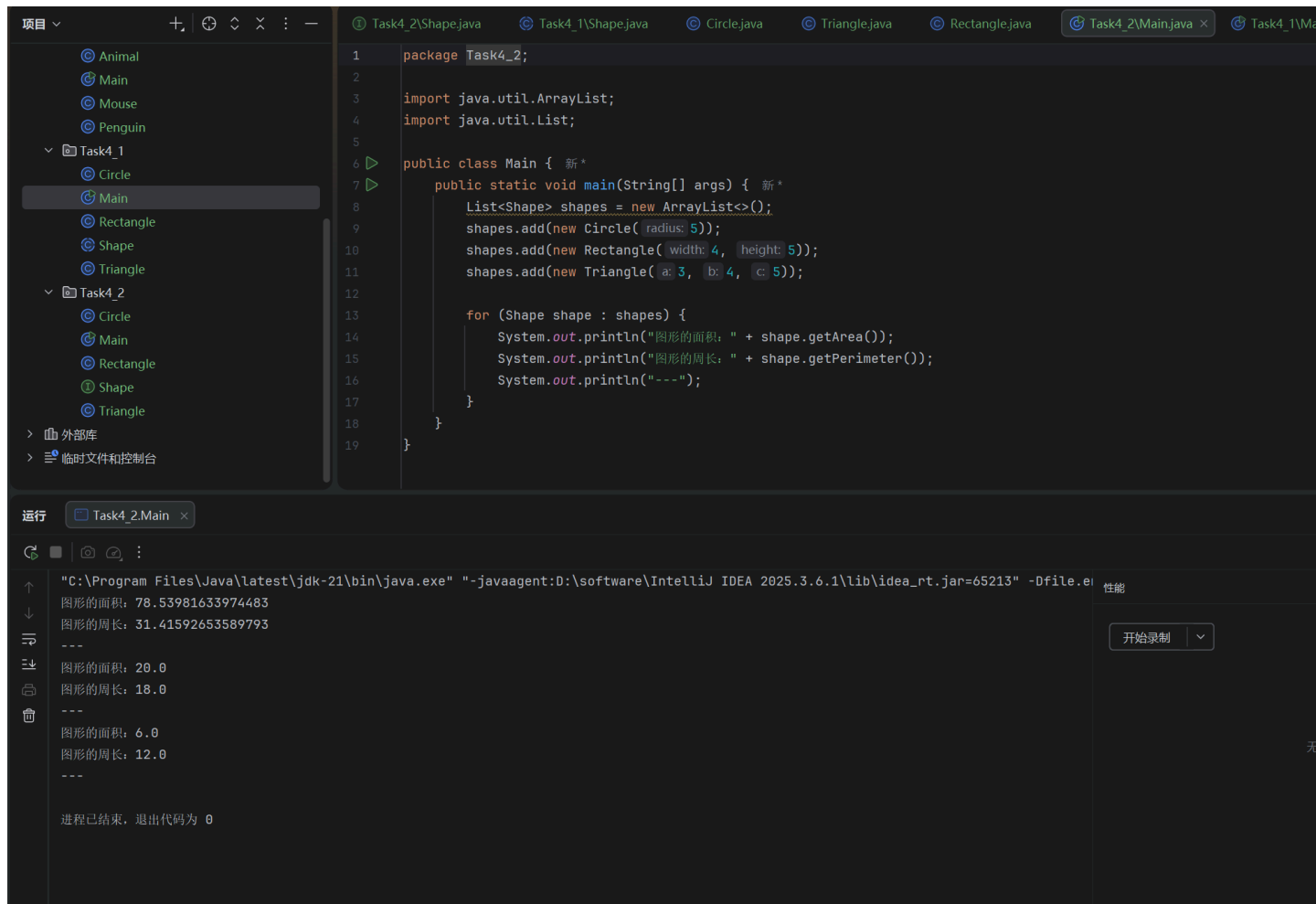
"C:\Program Files\Java\latest\jdk-21\bin\java.exe" "-javaagent:D:\software\IntelliJ IDEA 2025.3.6.1\lib\idea_rt.jar=59955" -Dfile.encoding=UTF-8

图形的面积: 78.53981633974483
图形的周长: 31.41592653589793

图形的面积: 20.0
图形的周长: 18.0

图形的面积: 6.0
图形的周长: 12.0

进程已结束，退出代码为 0



(3) 区别：抽象适用于是一类事物如都是图形，有共同属性可以用extends继承，抽象类里可写具体方法给子类复

接口适用于都能做同一件事，比如都能算面积周长，借此将多物联系起来，统一调用，用implements（像是合约关系，此时一个类可以有多个接口，因为它可以有很多能力），接口shape不写class，因为并没有说他们是一类

Task5

(1)

- private：balance和password用private保护：外部不能看见密码，也不能修改余额，保障资金安全同时validatePassword () validateAmount () 是内部工具，外界无需知道，用private隐藏
- public：安全暴露窗口，比如deposit ()，withdraw ()，transfer ()，所有操作需经过这些方法，内部会检验，金额大于0，密码需正确

考虑实际用户需求流程，密码、查询、金额安全性、私密性、余额和存取款金额数值大小等